



Digital Transformation

RELATÓRIO FINAL DE EQUIPE

Trabalho em Equipe com GitFlow e Validação de Qualidade — Portal Fake

Técnicas de Hyperautomation

Equipe 04 | **Prof. Moisés Levy**

Rômulo da Silva Lira | Huan Cruz de Oliveira

Gilvan Daniel da Silva | Janaina dos Santos Ferreira

Versão: 1.0 | **Data: 07/07/2026**

Histórico de Revisões

Versão	Data	Responsável	Descrição da Alteração	Status
1.0	07/07/2026	Equipe 04	Criação inicial do relatório, com a divisão de papéis da equipe	Aprovado



Digital Transformation

Sumário

Histórico de Revisões	1
1 Introdução	3
2 Equipe e Divisão de Papéis	3
3 O que Cada Aluno Fez	3
3.1 Rômulo da Silva Lira (Aluno A) — Líder Técnico / Cadastro	3
3.2 Huan Cruz de Oliveira (Aluno B) — Consulta de Usuários	4
3.3 Gilvan Daniel da Silva (Aluno C) — Exportação de Dados	6
3.4 Janaina dos Santos Ferreira (Aluno D) — Relatório Final	8
4 Como Foi Feito — Fluxo GitFlow Aplicado	9
5 Problemas Encontrados	9
5.1 Configuração incorreta de autoria no Git	9
5.2 Python não reconhecido no terminal	9
6 Sugestões de Melhoria	9
7 Avaliação Final da Solução	10



Digital Transformation

1 Introdução

Este relatório consolida as atividades desenvolvidas pela Equipe 04 na disciplina de Técnicas de Hyperautomation, referentes ao Roteiro 06 — Trabalho em Equipe com GitFlow e Validação de Qualidade no Portal Fake. O objetivo da atividade foi simular um ambiente corporativo real de desenvolvimento colaborativo, aplicando o fluxo **GitFlow** (feature → develop → main) na construção de módulos de automação para o sistema **Portal Fake**, com uso de **Git** e **GitHub** para versionamento e integração via Pull Requests.

Repositório da equipe: <https://github.com/CodeMartell/project-test>

2 Equipe e Divisão de Papéis

A divisão de responsabilidades seguiu o modelo proposto no roteiro, com cada integrante atuando em sua própria *feature branch*:

Papel	Integrante	Branch	Responsabilidade
Aluno A	Rômulo da Silva Lira	feature/cadastro	Líder técnico e Cadastro de Usuários
Aluno B	Huan Cruz de Oliveira	feature/consulta	Consulta de Usuários
Aluno C	Gilvan Daniel da Silva	feature/exportacao	Exportação de Dados
Aluno D	Janaina dos Santos Ferreira	feature/relatorio	Relatório Final da Equipe

3 O que Cada Aluno Fez

3.1 Rômulo da Silva Lira (Aluno A) — Líder Técnico / Cadastro

Branch: feature/cadastro

Objetivo: Desenvolver o módulo responsável pelo cadastro de usuários do Portal Fake, além de estruturar e organizar o repositório da equipe no GitHub.

Criou e organizou o repositório da equipe no GitHub (project-test), estruturando as branches *main* e *develop* conforme o fluxo GitFlow, adicionou os demais integrantes como colaboradores e compartilhou o link do projeto.

Desenvolveu, na branch *feature/cadastro*, o módulo `cadastro.py`, responsável por coletar os dados do novo usuário via terminal, validá-los (nome e sobrenome obrigatórios, CPF com exatamente 11 dígitos numéricos, e-mail com formato básico válido e verificação de duplicidade de CPF) e inserir o registro na planilha `Dados do Roteiro 06.xlsx` (aba *Usuarios*), utilizando a biblioteca `openpyxl`.

```
1 import os
2 import openpyxl
3
4 ARQUIVO_XLSX = os.path.join(os.path.dirname(os.path.abspath(__file__)), "Dados do Roteiro 06.
  xlsx")
5 NOME_PLANILHA = "Usuarios"
6
7 def cadastrar_usuario():
8     """
9     Solicita os dados do usuario via terminal e insere uma nova linha
```

```

10 na planilha 'Usuarios' do arquivo 'Dados do Roteiro 06.xlsx',
11 respeitando a estrutura de colunas existente:
12     A: Nome | B: Sobrenome | C: CPF | D: E-mail | E: Telefone
13     F: Nascimento | G: Status | H: Observacao | I: Endereco
14     ""
15     nome = input("Nome: ").strip()
16     sobrenome = input("Sobrenome: ").strip()
17     cpf = input("CPF (somente numeros): ").strip()
18     email = input("E-mail: ").strip()
19     telefone = input("Telefone (somente numeros): ").strip()
20     nascimento = input("Data de Nascimento (DD/MM/AAAA): ").strip()
21
22     while True:
23         status = input("Status (ATIVO / INATIVO): ").strip().upper()
24         if status in ("ATIVO", "INATIVO"):
25             break
26         print("Valor invalido. Digite ATIVO ou INATIVO.")
27
28     observacao = input("Observacao: ").strip()
29     endereco = input("Endereco: ").strip()
30
31     if not nome or not sobrenome:
32         print("Nome e Sobrenome sao obrigatorios. Cadastro cancelado.")
33         return
34     if len(cpf) != 11 or not cpf.isdigit():
35         print("CPF deve conter exatamente 11 digitos numericos. Cadastro cancelado.")
36         return
37     if "@" not in email or "." not in email:
38         print("E-mail invalido. Cadastro cancelado.")
39         return
40
41     try:
42         wb = openpyxl.load_workbook(ARQUIVO_XLSX)
43     except FileNotFoundError:
44         print(f"Arquivo '{ARQUIVO_XLSX}' nao encontrado.")
45         return
46
47     ws = wb[NOME_PLANILHA]
48
49     for row in ws.iter_rows(min_row=2, max_col=3, values_only=True):
50         if row[2] == cpf:
51             print(f"CPF {cpf} ja cadastrado. Cadastro cancelado.")
52             wb.close()
53             return
54
55     nova_linha = [nome, sobrenome, cpf, email, telefone, nascimento, status, observacao,
56                  endereco]
57     ws.append(nova_linha)
58     wb.save(ARQUIVO_XLSX)
59     wb.close()
60     print(f"Usuario cadastrado com sucesso! {nome} {sobrenome} - CPF: {cpf}")
61
62 if __name__ == "__main__":
63     cadastrar_usuario()

```

3.2 Huan Cruz de Oliveira (Aluno B) — Consulta de Usuários

Branch: feature/consulta

Objetivo: Desenvolver o módulo responsável pela consulta de usuários cadastrados no Portal Fake. Desenvolveu, na branch feature/consulta, o módulo consulta.py, responsável pela automação da con-

sulta de usuários cadastrados no Portal Fake. O escopo cobriu três funcionalidades obrigatórias: pesquisa de usuários por múltiplos critérios (nome ou CPF), exibição estruturada dos metadados encontrados e validação de consistência da consulta.

Ferramentas e tecnologias: Visual Studio Code (edição e terminal integrado); Python 3.x; e a biblioteca `openpyxl` para leitura da planilha de dados, escolhida no lugar de bibliotecas mais pesadas como o Pandas por permitir uma varredura direta e leve das células do Excel.

Fonte de dados: o script lê diretamente a planilha `Dados do Roteiro 06.xlsx`, contendo os registros oficiais de teste (nome, sobrenome, CPF, e-mail, telefone, status e endereço de 10 usuários), comparando o termo pesquisado com o nome completo (busca por fragmento) ou com o CPF (busca exata).

```
1 import openpyxl
2
3
4 def consultar_usuario():
5     print("=" * 45)
6     print("SISTEMA DE CONSULTA DE USUARIOS - RPA")
7     print("=" * 45)
8
9     nome_arquivo = "Dados do Roteiro 06.xlsx"
10    try:
11        # Abre o arquivo Excel de forma nativa
12        wb = openpyxl.load_workbook(nome_arquivo, data_only=True)
13        # Seleciona a primeira planilha ativa
14        aba = wb.active
15    except FileNotFoundError:
16        print(f"\nErro de Validacao: O arquivo '{nome_arquivo}' nao foi encontrado.")
17        return
18
19    # Entrada para pesquisa de usuarios
20    pesquisa = input("Digite o Nome ou CPF para pesquisar: ").strip().lower()
21    encontrado = False
22
23    # Percorre as linhas do Excel (comecando da linha 2 para pular o cabecalho)
24    for linha in aba.iter_rows(min_row=2, max_row=aba.max_row, values_only=True):
25        # Se a linha estiver vazia, pula
26        if not linha[0]:
27            continue
28
29        # Mapeamento das colunas com base na estrutura do roteiro
30        nome = str(linha[0]).strip()
31        sobrenome = str(linha[1]).strip()
32        cpf = str(linha[2]).strip()
33        email = str(linha[3]).strip()
34        telefone = str(linha[4]).strip()
35        nascimento = str(linha[5]).strip()
36        status = str(linha[6]).strip()
37        observacao = str(linha[7]).strip()
38        endereco = str(linha[8]).strip()
39        nome_completo = f"{nome} {sobrenome}"
40
41        # Critério de busca (por parte do nome ou CPF exato)
42        if (pesquisa in nome_completo.lower()) or (pesquisa == cpf):
43            print("\nUsuario Localizado com Sucesso!")
44            print(f"Nome Completo : {nome_completo}")
45            print(f"CPF : {cpf}")
46            print(f"E-mail : {email}")
47            print(f"Telefone : {telefone}")
48            print(f"Nascimento : {nascimento}")
49            print(f>Status : {status}")
50            print(f"Endereco : {endereco}")
51            print(f"Observacao : {observacao}")
```

```

52         print("-" * 45)
53         encontrado = True
54
55     if not encontrado:
56         print("\nRegistro nao encontrado na base de dados atual.")
57
58
59 if __name__ == "__main__":
60     consultar_usuario()

```

3.3 Gilvan Daniel da Silva (Aluno C) — Exportação de Dados

Branch: feature/exportacao

Objetivo: Desenvolver o módulo responsável pela exportação dos dados do Portal Fake.

Desenvolveu, na branch feature/exportacao, o módulo `exportacao.py`, responsável por simular a geração e o download do arquivo de exportação dos registros do Portal Fake:

```

1  import csv
2  from datetime import datetime
3  from pathlib import Path
4  from typing import Any, Dict, List, Optional
5
6
7  def _obter_colunas(registros: List[Dict[str, Any]]) -> List[str]:
8      """
9      Identifica todas as colunas existentes nos registros,
10     preservando a ordem em que foram encontradas.
11     """
12     colunas: List[str] = []
13
14     for registro in registros:
15         for coluna in registro.keys():
16             if coluna not in colunas:
17                 colunas.append(coluna)
18
19     return colunas
20
21
22 def _normalizar_valor(valor: Any) -> str:
23     """
24     Converte valores para um formato adequado ao arquivo CSV.
25     """
26     if valor is None:
27         return ""
28
29     if isinstance(valor, bool):
30         return "Sim" if valor else "Não"
31
32     texto = str(valor)
33
34     # Evita que programas de planilha interpretem o valor como fórmula.
35     if texto.startswith(("=", "+", "-", "@")):
36         return "'" + texto
37
38     return texto
39
40
41 def exportar_dados(
42     registros: List[Dict[str, Any]],
43     nome_arquivo: Optional[str] = None,

```

```

44     diretorio_saida: str = "exportacoes",
45 ) -> Optional[Path]:
46     """
47     Exporta uma lista de registros para um arquivo CSV.
48
49     Parâmetros:
50         registros:
51             Lista de dicionários contendo os dados dos usuários.
52
53         nome_arquivo:
54             Nome opcional do arquivo CSV.
55
56         diretorio_saida:
57             Pasta onde o arquivo será gerado.
58
59     Retorno:
60         Caminho do arquivo gerado ou None quando não houver exportação.
61     """
62
63     if not registros:
64         print("Exportação não realizada: nenhum registro disponível.")
65         return None
66
67     if not all(isinstance(registro, dict) for registro in registros):
68         raise TypeError(
69             "Cada registro deve ser representado por um dicionário."
70         )
71
72     colunas = _obter_colunas(registros)
73
74     if not colunas:
75         print(
76             "Exportação não realizada: "
77             "os registros não possuem campos."
78         )
79         return None
80
81     pasta_saida = Path(diretorio_saida)
82     pasta_saida.mkdir(parents=True, exist_ok=True)
83
84     if nome_arquivo is None:
85         horario = datetime.now().strftime("%Y%m%d_%H%M%S")
86         nome_arquivo = f"usuarios_{horario}.csv"
87     elif not nome_arquivo.lower().endswith(".csv"):
88         nome_arquivo += ".csv"
89
90     caminho_saida = pasta_saida / nome_arquivo
91
92     try:
93         with caminho_saida.open(
94             mode="w",
95             newline="",
96             encoding="utf-8-sig",
97         ) as arquivo:
98             escritor = csv.DictWriter(
99                 arquivo,
100                 fieldnames=colunas,
101                 delimiter=";",
102                 extrasaction="ignore",
103                 restval="",
104             )
105
106             escritor.writeheader()

```



```

107         for registro in registros:
108             registro_normalizado = {
109                 coluna: _normalizar_valor(
110                     registro.get(coluna)
111                 )
112             }
113             for coluna in colunas
114         }
115
116         escritor.writerow(registro_normalizado)
117
118     except OSError as erro:
119         print(f"Falha ao gerar o arquivo CSV: {erro}")
120         return None
121
122     print(
123         "Exportação concluída com sucesso: "
124         f"{caminho_saida.resolve()}"
125     )
126     print(
127         f"Total de registros exportados: {len(registros)}"
128     )
129
130     return caminho_saida
131
132
133 if __name__ == "__main__":
134     usuarios_exemplo = [
135         {
136             "nome": "Ana Souza",
137             "cpf": "12345678900",
138             "email": "ana.souza@email.com",
139             "ativo": True,
140         },
141         {
142             "nome": "Carlos Lima",
143             "cpf": "98765432100",
144             "email": "carlos.lima@email.com",
145             "ativo": False,
146         },
147     ]
148
149     print("Teste 1: exportação com registros")
150     exportar_dados(usuarios_exemplo)
151
152     print("\nTeste 2: exportação sem registros")
153     exportar_dados([])

```

Além do desenvolvimento do módulo, identificou e corrigiu um problema de configuração do Git na máquina utilizada em laboratório, referente à autoria dos commits (detalhado na Seção 5).

3.4 Janaina dos Santos Ferreira (Aluno D) — Relatório Final

Branch: feature/relatorio

Objetivo: Consolidar as entregas da equipe e elaborar o relatório final, documentando o que foi feito por cada integrante, como foi feito, os problemas encontrados e as sugestões de melhoria.

Ficou responsável por reunir as informações repassadas pelos demais integrantes (Rômulo, Huan e Gilvan) e elaborar o presente relatório final da equipe.

4 Como Foi Feito — Fluxo GitFlow Aplicado

A equipe seguiu o fluxo GitFlow orientado em sala, com a seguinte sequência de trabalho:

1. O Aluno A criou o repositório no GitHub e inicializou as branches `main` e `develop`;
2. Cada integrante clonou o repositório e criou sua própria branch de funcionalidade a partir da `develop` (`feature/cadastro`, `feature/consulta`, `feature/exportacao` e `feature/relatorio`), desenvolvendo seu módulo/entrega de forma independente;
3. Cada módulo foi versionado com `git add`, `git commit -m` e `git push origin feature/[nome-da-branch]`;
4. Ao final do desenvolvimento, foram abertos Pull Requests de cada branch `feature/*` para a branch `develop`;
5. Os Pull Requests foram revisados e integrados (*merge*) à branch `develop`, unificando o trabalho de todos os integrantes;
6. Um ajuste adicional foi realizado por meio de uma branch de correção (`revert-2-feature/exportacao`), com Pull Request próprio, para corrigir um problema identificado no módulo de exportação antes da integração final.

5 Problemas Encontrados

5.1 Configuração incorreta de autoria no Git

No computador utilizado em laboratório, o Git estava previamente configurado com as credenciais de outra pessoa, fazendo com que o primeiro commit do módulo de exportação (Aluno C) aparecesse atribuído a um autor diferente do real.

Atenção

Solução aplicada: o problema foi corrigido reconfigurando as credenciais locais do Git e corrigindo a autoria do commit já existente, sem a necessidade de refazer o trabalho já desenvolvido.

```
git config user.name "Gilvan Daniel da Silva"
git config user.email "gilvandnel@gmail.com"
git commit --amend --reset-author --no-edit
```

5.2 Python não reconhecido no terminal

A máquina utilizada não reconhecia o comando `python` no terminal, impedindo a verificação inicial do ambiente. Foi necessário reinstalar/configurar corretamente o Python (e suas variáveis de ambiente) antes de dar continuidade ao desenvolvimento do módulo.

6 Sugestões de Melhoria

- Executar a etapa de "Setup Técnico" do roteiro (`python --version`, `pip --version` e `git --version`) antes de iniciar qualquer commit, especialmente em máquinas de laboratório compartilhadas;
- Sempre conferir `git config --global user.name` e `git config --global user.email` no início do trabalho, principalmente em computadores usados por outras pessoas anteriormente;

- Definir previamente qual máquina cada integrante utilizará, para evitar o retrabalho de remover configurações de contas de terceiros;
- Manter o uso de Pull Requests mesmo para correções pontuais (como feito na branch de revert), documentando o motivo do ajuste na descrição do PR para facilitar a revisão.

7 Avaliação Final da Solução

A equipe conseguiu aplicar corretamente o fluxo GitFlow (feature → develop → main), com cada integrante trabalhando de forma independente em sua própria branch e integrando o código por meio de Pull Requests revisados antes do merge para a branch develop. Mesmo diante de imprevistos técnicos de ambiente (configuração de Git e instalação do Python) e de um pequeno problema de qualidade de código identificado em revisão, a equipe conseguiu identificar rapidamente a causa dos problemas, corrigi-los sem perda de trabalho já realizado e manter a organização do repositório, demonstrando maturidade no uso de versionamento colaborativo em um cenário próximo ao de um ambiente corporativo real.

fim do capítulo ■



Digital Transformation



Digital Transformation